Comparando complexidade e tempo de algoritmos de ordenação

Bruno Solari[1], **Thales Kun**[1]

[1]Curso de Engenharia de Computação – Universidade Federal do Pampa (UNIPAMPA)
Bagé – RS – Brasil

{brunosolari.aluno,thalessales.aluno}@unipampa.edu.br

Abstract. *This work aims to analyze and compare the performance of ten sorting algorithms, relating their theoretical complexities to the results obtained through practical experiments. To achieve this, tests were conducted on two computers with different hardware configurations, using arrays of different sizes and arrangements. The analysis seeks to demonstrate how an increase in the number of elements affects the execution time of the algorithms, highlighting the performance differences between them. Thus, this work aims to emphasize the importance of choosing appropriate algorithms for different applications, allowing readers to understand their main characteristics through a combination of theoretical concepts and experimental results. The experiments showed that quadratic algorithms may require substantially more time for large inputs, while the initial ordering of the data also affects performance.*

Resumo. *Este trabalho tem como objetivo analisar e comparar o desempenho de dez algoritmos de ordenação, relacionando suas complexidades teóricas aos resultados obtidos por meio de experimentos práticos. Para isso, foram realizados testes em dois computadores com configurações distintas, utilizando vetores de diferentes tamanhos e organizações. A análise busca demonstrar como o aumento da quantidade de elementos influencia o tempo de execução dos algoritmos, destacando as diferenças de desempenho entre eles. Dessa forma, o trabalho procura evidenciar a importância da escolha de algoritmos adequados para diferentes aplicações, permitindo ao leitor compreender suas principais características por meio da combinação entre fundamentos teóricos e resultados experimentais. Observou-se que algoritmos de complexidade quadrática podem apresentar tempos muito elevados para entradas maiores e que a organização inicial dos dados influencia o desempenho.*

1. Introdução

Os algoritmos de ordenação são fundamentais na computação, pois permitem organizar dados de maneira eficiente, facilitando sua manipulação e utilização em diferentes aplicações. Entretanto, cada algoritmo possui características e complexidades distintas, que podem influenciar diretamente seu desempenho.

Neste trabalho, foram implementados e analisados dez algoritmos de ordenação em Java, utilizando vetores de diferentes tamanhos e organizações. Os experimentos foram realizados em dois computadores com configurações distintas, permitindo observar as diferenças nos tempos de execução.

O objetivo principal é relacionar os resultados experimentais às complexidades teóricas dos algoritmos, demonstrando a importância de escolher métodos adequados conforme as características e a quantidade de dados. A análise dos algoritmos de ordenação e de suas complexidades apoia-se em referências clássicas da área [1, 2].

2. Algoritmos analisados

Os dez algoritmos de ordenação analisados neste trabalho foram selecionados com base nos conteúdos abordados em aula. O objetivo é consolidar o entendimento sobre seu funcionamento e relacionar suas características teóricas aos resultados obtidos experimentalmente. Além disso, busca-se compreender como as complexidades de melhor, médio e pior caso influenciam o desempenho dos algoritmos em diferentes situações.

2.1. Algoritmos de comparação

Os algoritmos baseados em comparação realizam a ordenação dos elementos por meio de comparações entre seus valores. Apesar de utilizarem o mesmo princípio, apresentam diferentes estratégias de ordenação e complexidades computacionais [2, 1].

2.1.1. Bubble Sort

O Bubble Sort é um algoritmo simples que percorre o vetor comparando elementos adjacentes e realizando trocas quando estão fora de ordem. Esse processo é repetido até que todos os elementos estejam ordenados.

Sua complexidade no caso médio e no pior caso é de $O(n^2)$. Entretanto, quando implementado com uma verificação de trocas, pode atingir $O(n)$ no melhor caso, quando o vetor já está ordenado.

2.1.2. Insertion Sort

O Insertion Sort funciona construindo uma sequência ordenada gradualmente. A cada iteração, um elemento é selecionado e inserido na posição correta entre os elementos já ordenados.

Apresenta complexidade $O(n)$ no melhor caso, quando os elementos já estão ordenados, e $O(n^2)$ nos casos médio e pior. É especialmente eficiente para vetores pequenos ou parcialmente ordenados.

2.1.3. Selection Sort

O Selection Sort divide o vetor em uma parte ordenada e outra não ordenada. A cada passagem, identifica o menor elemento da parte não ordenada e o posiciona na posição correspondente da parte ordenada.

Sua complexidade é $O(n^2)$ nos casos melhor, médio e pior, pois continua realizando as comparações necessárias para encontrar o menor elemento, independentemente da organização inicial do vetor.

2.1.4. Shell Sort

O Shell Sort é uma evolução do Insertion Sort, que realiza comparações entre elementos separados por determinados intervalos, conhecidos como gaps. Esses intervalos são reduzidos progressivamente até que a última etapa utilize um gap de uma posição.

A implementação utilizada empregou a sequência de Knuth, com intervalos $1, 4, 13, 40, 121, \ldots$, gerados pela relação $h = 3h + 1$ e reduzidos até $h = 1$. A complexidade do Shell Sort depende da sequência de intervalos escolhida, portanto não pode ser caracterizada apenas por uma única ordem de crescimento independente da implementação.

2.1.5. Heap Sort

O Heap Sort utiliza uma estrutura de dados chamada heap, que permite organizar os elementos de forma hierárquica. Inicialmente, constrói um heap e, posteriormente, remove repetidamente o maior elemento, reorganizando a estrutura até completar a ordenação.

Apresenta complexidade $O(n \log n)$ nos casos melhor, médio e pior, mantendo um desempenho assintótico consistente independentemente da organização inicial dos elementos.

2.1.6. Merge Sort

O Merge Sort utiliza a estratégia de divisão e conquista, dividindo o vetor em partes menores até obter sequências com apenas um elemento. Posteriormente, essas partes são combinadas de maneira ordenada até reconstruir o vetor completo.

Sua complexidade é $O(n \log n)$ nos casos melhor, médio e pior. Entretanto, sua implementação tradicional utiliza memória auxiliar para realizar a intercalação dos elementos.

2.1.7. Quick Sort

O Quick Sort também utiliza a estratégia de divisão e conquista. Seu funcionamento consiste em selecionar um elemento denominado pivô e reorganizar o vetor, posicionando elementos menores e maiores em lados opostos da partição. O processo é aplicado recursivamente às partes resultantes.

Apresenta complexidade $O(n \log n)$ nos casos melhor e médio, podendo atingir $O(n^2)$ no pior caso, quando as partições ficam muito desequilibradas. A estratégia de escolha do pivô influencia diretamente seu desempenho.

2.2. Algoritmos de tempo linear

Diferentemente dos algoritmos anteriores, os algoritmos desta categoria não dependem exclusivamente de comparações entre os elementos. Eles utilizam características específicas dos valores, como sua faixa de distribuição ou representação numérica, podendo alcançar complexidades lineares sob determinadas condições [1].

2.2.1. Counting Sort

O Counting Sort realiza a ordenação por meio da contagem das ocorrências de cada valor presente no vetor. A partir dessas contagens, determina as posições dos elementos na sequência ordenada.

Sua complexidade é $O(n + k)$, sendo n a quantidade de elementos e k o tamanho da faixa de valores possíveis. É eficiente quando essa faixa é relativamente pequena, mas pode apresentar elevado consumo de memória quando os valores estão distribuídos em um intervalo muito grande.

2.2.2. Radix Sort

O Radix Sort ordena os elementos considerando seus dígitos ou posições individuais. O processo ocorre em etapas, utilizando uma ordenação estável para organizar os elementos de acordo com cada dígito.

Sua complexidade é $O(d(n+b))$, em que d representa a quantidade de dígitos e b a base utilizada. Pode apresentar desempenho linear quando esses parâmetros são limitados em relação à quantidade de elementos.

2.2.3. Bucket Sort

O Bucket Sort distribui os elementos em diferentes grupos, denominados buckets, de acordo com seus valores. Posteriormente, cada grupo é ordenado individualmente e os resultados são reunidos para formar o vetor ordenado.

Considerando uma distribuição aproximadamente uniforme dos elementos, sua complexidade esperada pode ser $O(n + k)$, sendo k o número de buckets. Entretanto, dependendo da distribuição dos valores e do método utilizado para ordenar cada grupo, o pior caso pode atingir $O(n^2)$.

3. Metodologia

Para avaliar o desempenho dos dez algoritmos de ordenação, foram realizados experimentos utilizando vetores de diferentes tamanhos e organizações. Os testes ocorreram em dois computadores com configurações distintas, permitindo observar o comportamento dos algoritmos em diferentes ambientes de execução.

3.1. Ambientes de execução

Os experimentos foram realizados nos seguintes computadores:

Computador 1:

- Modelo: Lenovo IdeaPad 3 15ITL6;
- Processador: Intel Core i5-1135G7, 11ª geração;
- Memória RAM: 8 GB;
- Sistema operacional: Zorin OS 18.1 Core;
- Java: OpenJDK 25.0.2 LTS (Eclipse Temurin, 64 bits).

Computador 2:

- Processador: AMD Ryzen 7 5700G;
- Memória RAM: 16 GB;
- Placa de vídeo: AMD Radeon RX 6600;
- Sistema operacional: Windows 11;
- Ambiente de execução: Ubuntu Linux por meio do WSL (Windows Subsystem for Linux);
- Java: OpenJDK 25.0.2 LTS.

O WSL permite executar um ambiente Linux dentro do Windows, possibilitando a utilização de ferramentas e programas desenvolvidos para esse sistema.

É importante destacar que as diferenças de desempenho entre os computadores não dependem exclusivamente do processador, pois fatores como memória, sistema operacional e Máquina Virtual Java (JVM) também podem influenciar os tempos de execução.

3.2. Preparação dos dados de entrada

Os testes foram realizados utilizando vetores com quatro tamanhos diferentes:

- 1.000 elementos;
- 10.000 elementos;
- 100.000 elementos;
- 1.000.000 de elementos.

Para cada tamanho, foram consideradas três organizações iniciais dos elementos:

- **Crescente:** elementos organizados do menor para o maior;
- **Decrescente:** elementos organizados do maior para o menor;
- **Aleatório:** elementos distribuídos em ordem aleatória.

Os vetores foram gerados automaticamente pelo programa de experimentação. Para os dados aleatórios, foi utilizada uma semente fixa, permitindo reproduzir a mesma sequência de valores. O gerador adotou a semente base `20261020`, somada ao tamanho do vetor, e valores inteiros não negativos entre zero e dez vezes o tamanho do vetor. Nos casos crescentes, foram gerados valores de zero até $n - 1$; nos decrescentes, de n até um.

Além disso, cada algoritmo recebeu uma cópia do vetor original correspondente à configuração testada. Dessa forma, a execução de um método não alterava os dados utilizados pelos demais algoritmos.

3.3. Verificação da corretude

Antes dos experimentos de desempenho, foi executado o programa auxiliar `TesteCorretude.java`, que verifica as saídas dos dez algoritmos em vetores pequenos. Foram utilizados casos crescentes, decrescentes, aleatórios e com elementos repetidos, incluindo tamanhos pares (12) e ímpares (13 e 15, conforme o caso).

Em cada verificação, o resultado do algoritmo foi comparado com um vetor de referência ordenado por `Arrays.sort()`, utilizado exclusivamente como gabarito de teste, sem substituir as implementações avaliadas. Foram realizadas 63 verificações dos sete algoritmos por comparação e 24 dos três não comparativos, totalizando 87 testes. Todas as verificações foram aprovadas na execução realizada.

A Figura 1 apresenta trechos dos registros dos testes por comparação e não comparativos, incluindo o resultado final. A aprovação desses casos constitui uma verificação prática, e não uma prova formal de corretude para todas as entradas possíveis.

```
=== Verificacao de corretude - TP1 ACD ===

--- Caso: Crescente (par, 12) ---
Entrada: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
  [OK] bubbleSort
  [OK] insertionSort
  [OK] selectionSort
  [OK] shellSort
  [OK] heapSort
  [OK] mergeSort
  [OK] quickSort

--- Caso: Decrescente (par, 12) ---
Entrada: [12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
  [OK] bubbleSort
  [OK] insertionSort
  [OK] selectionSort
  [OK] shellSort
  [OK] heapSort
  [OK] mergeSort
  [OK] quickSort
```

```
--- Caso: Linear - Decrescente (impar, 13) ---
Entrada: [13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
  [OK] countingSort
  [OK] radixSort
  [OK] bucketSort

--- Caso: Linear - Aleatorio (impar, 13) ---
Entrada: [36, 64, 85, 44, 80, 54, 68, 49, 50, 34, 0, 12, 8]
  [OK] countingSort
  [OK] radixSort
  [OK] bucketSort

--- Caso: Linear - Com repeticoes (impar, 13) ---
Entrada: [2, 2, 2, 0, 1, 1, 3, 2, 2, 2, 1, 3, 0]
  [OK] countingSort
  [OK] radixSort
  [OK] bucketSort


=== Resultado final: 87/87 testes OK ===
```

Figura 1. Trechos dos testes de corretude dos algoritmos por comparação e não comparativos; 87 verificações aprovadas.

3.4. Execução dos experimentos

Os dez algoritmos foram implementados em Java e executados por meio de um programa responsável por organizar os testes e registrar os tempos.

Para medir o desempenho, foi utilizado o método `System.nanoTime()`, que registra intervalos de tempo com valores expressos em nanossegundos, cuja resolução efetiva depende da plataforma [3].

A medição considerou apenas a execução do algoritmo de ordenação, sem incluir a geração dos vetores, a criação das cópias ou o armazenamento dos resultados. A comparação entre métodos de diferentes categorias também deve considerar que os algoritmos por comparação utilizaram `Integer[]`, enquanto os não comparativos utilizaram `int[]`.

Antes das medições, foram realizadas três execuções preliminares de cada configuração, com o objetivo de reduzir a influência do aquecimento inicial da JVM. Esses tempos não foram incluídos nos resultados.

3.5. Quantidade de execuções

Para reduzir a influência de variações ocasionais no desempenho, cada configuração foi executada múltiplas vezes.

No Computador 1, foram utilizadas 10 execuções por configuração nos resultados apresentados.

No Computador 2, foram originalmente realizadas 30 execuções para os vetores de até 100.000 elementos e 10 execuções para os vetores de 1.000.000 de elementos.

Para padronizar os gráficos, foram selecionadas as 10 primeiras execuções de cada configuração do Computador 2, mantendo os valores originais registrados.

Devido ao elevado tempo de processamento, os algoritmos Bubble Sort, Insertion Sort e Selection Sort não foram executados com 1.000.000 de elementos no Computador 1. Entretanto, esses testes foram realizados no Computador 2, permitindo observar seu comportamento com entradas maiores.

3.6. Organização e análise dos resultados

Os tempos individuais foram armazenados em arquivos CSV, contendo o algoritmo executado, o tamanho do vetor, a organização dos elementos, o número da repetição e o tempo registrado em nanossegundos.

Posteriormente, os dados foram processados utilizando Python, com auxílio das bibliotecas Pandas, NumPy e Matplotlib.

Foram calculadas medidas estatísticas, como média, mediana e tempo máximo, além da elaboração de gráficos de barras, crescimento e boxplots.

Em alguns gráficos foram utilizadas escalas logarítmicas, permitindo representar tempos com diferenças muito grandes de magnitude.

Por fim, os resultados foram relacionados às complexidades teóricas dos algoritmos, buscando compreender a influência do tamanho e da organização dos vetores sobre o desempenho observado.

4. Resultados

Esta seção apresenta os resultados dos experimentos conduzidos nos dois computadores. Os gráficos foram divididos entre crescimento por tamanho de entrada, tempos máximos com 100 mil elementos e tempos médios com 1 milhão de elementos. As comparações entre computadores correspondem a ambientes de execução diferentes e devem ser interpretadas com essa limitação.

4.1. Crescimento dos tempos de execução

As Figuras 2 a 5 apresentam o tempo mediano de execução em função do tamanho dos vetores, para as três organizações de entrada. Os eixos estão em escala logarítmica; a ausência de um ponto significa que aquela configuração não foi executada.

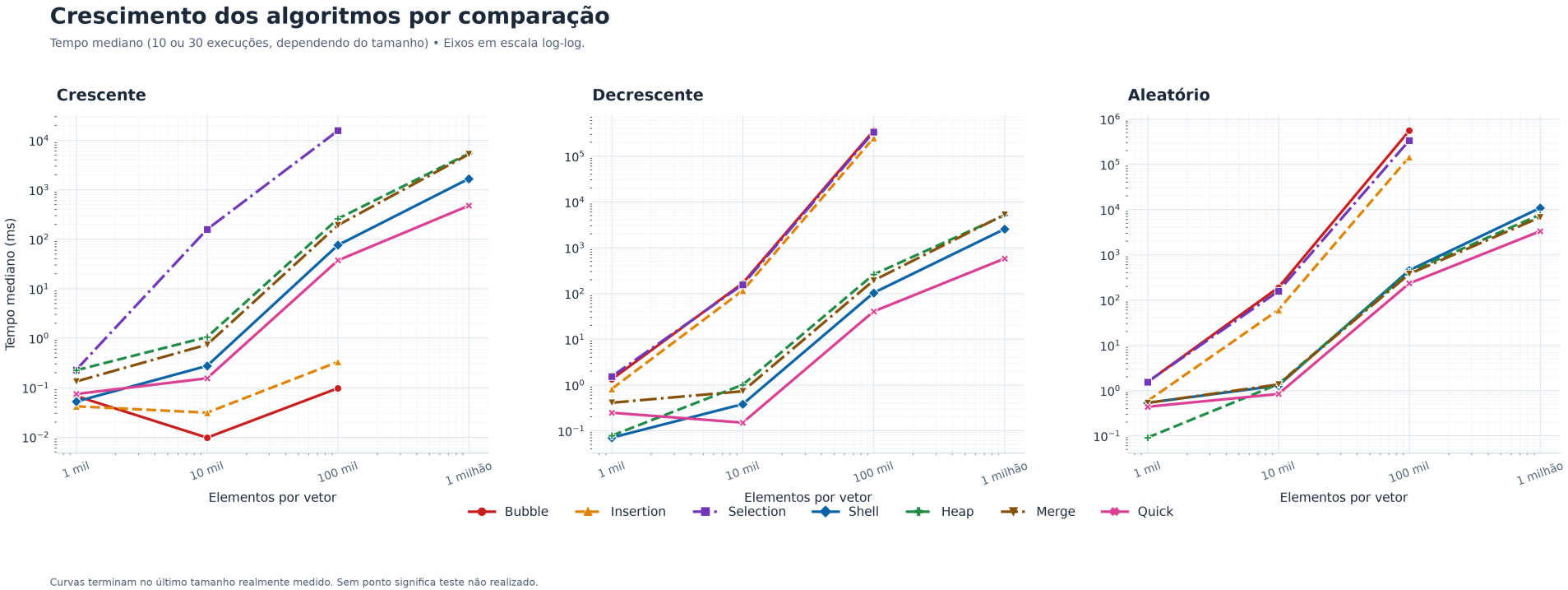


Figura 2. Crescimento dos tempos medianos dos algoritmos por comparação no Computador 1.

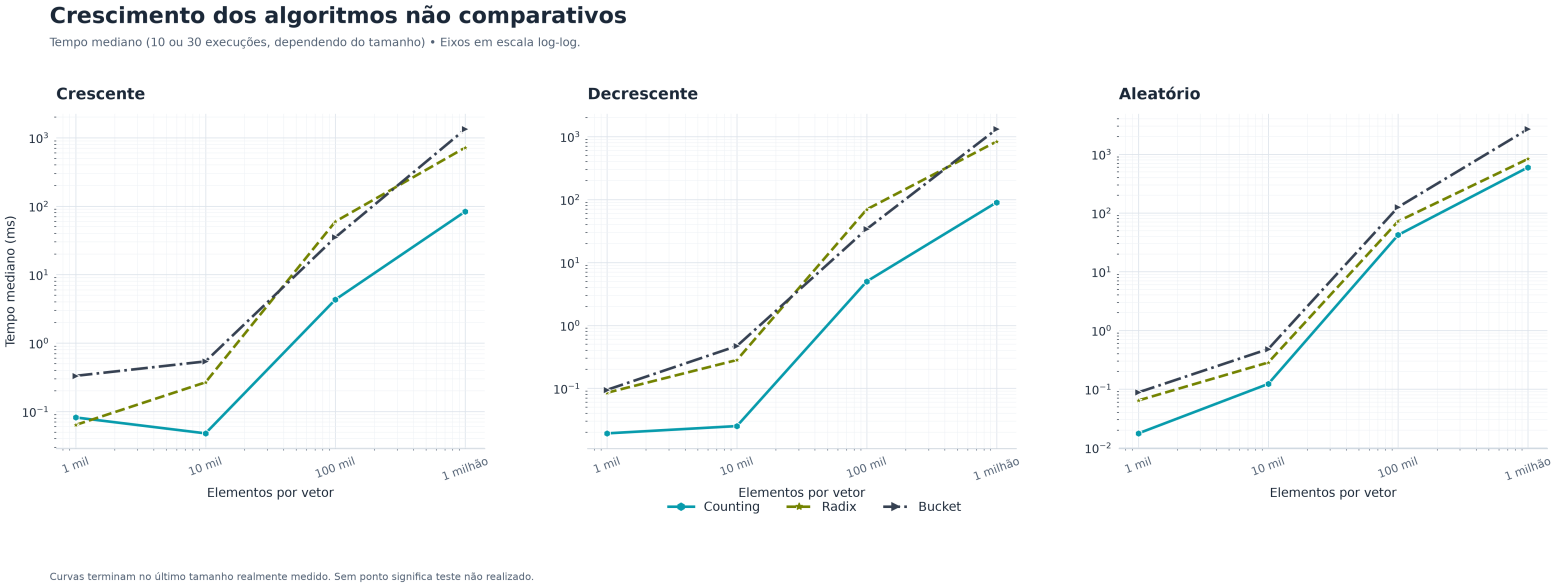


Figura 3. Crescimento dos tempos medianos dos algoritmos não comparativos no Computador 1.

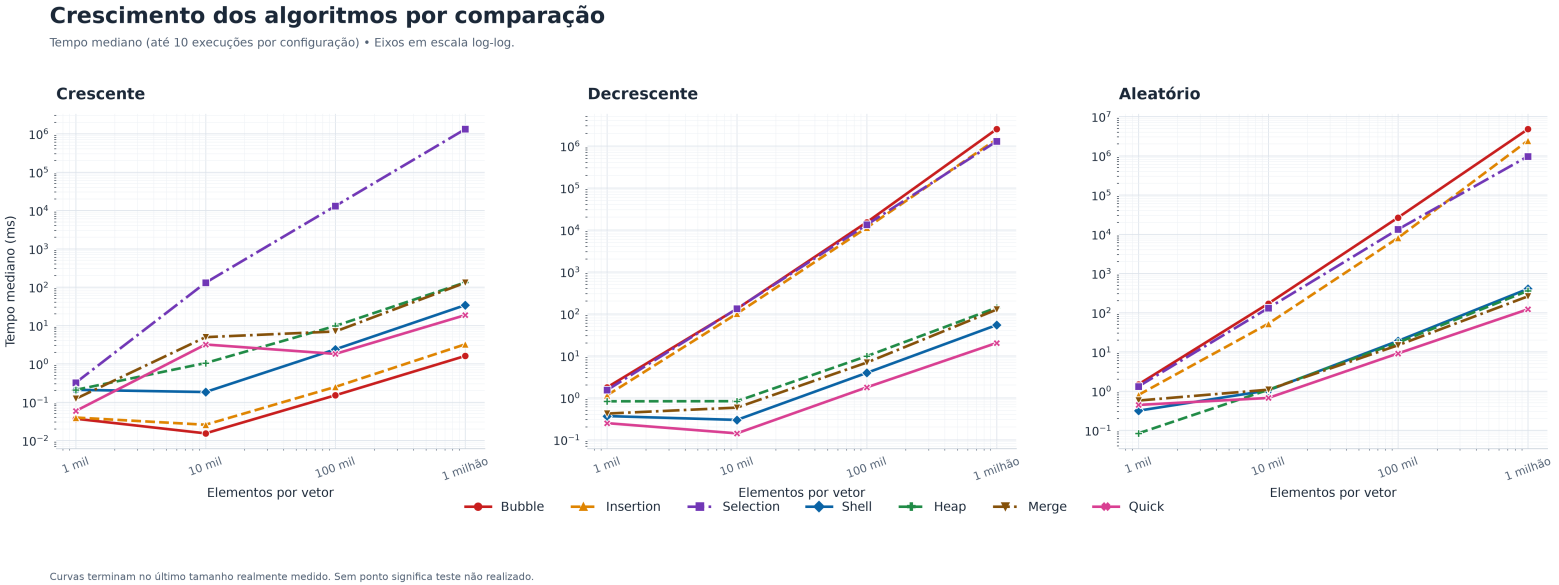


Figura 4. Crescimento dos tempos medianos dos algoritmos por comparação no Computador 2.

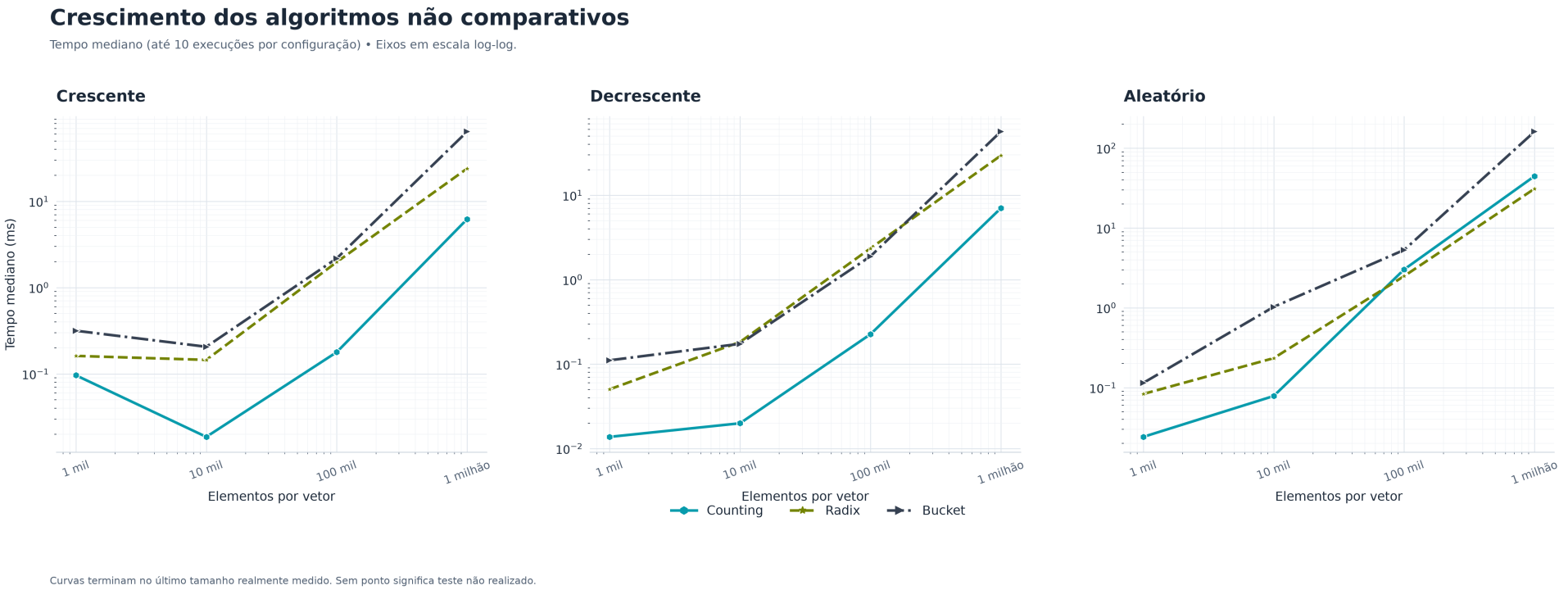


Figura 5. Crescimento dos tempos medianos dos algoritmos não comparativos no Computador 2.

A observação das curvas evidencia que os algoritmos de complexidade quadrática podem apresentar crescimento muito superior ao dos métodos de complexidade $O(n \log n)$. As diferenças entre Counting, Radix e Bucket também variam de acordo com a faixa de valores e a organização inicial dos vetores. A interpretação dos resultados é aprofundada na Seção 5.

4.2. Tempos máximos com 100 mil elementos

As Figuras 6 e 7 apresentam o maior tempo observado entre as dez execuções consideradas de cada algoritmo, nos cenários crescente, decrescente e aleatório. A escala logarítmica facilita a visualização de valores que diferem por várias ordens de grandeza.

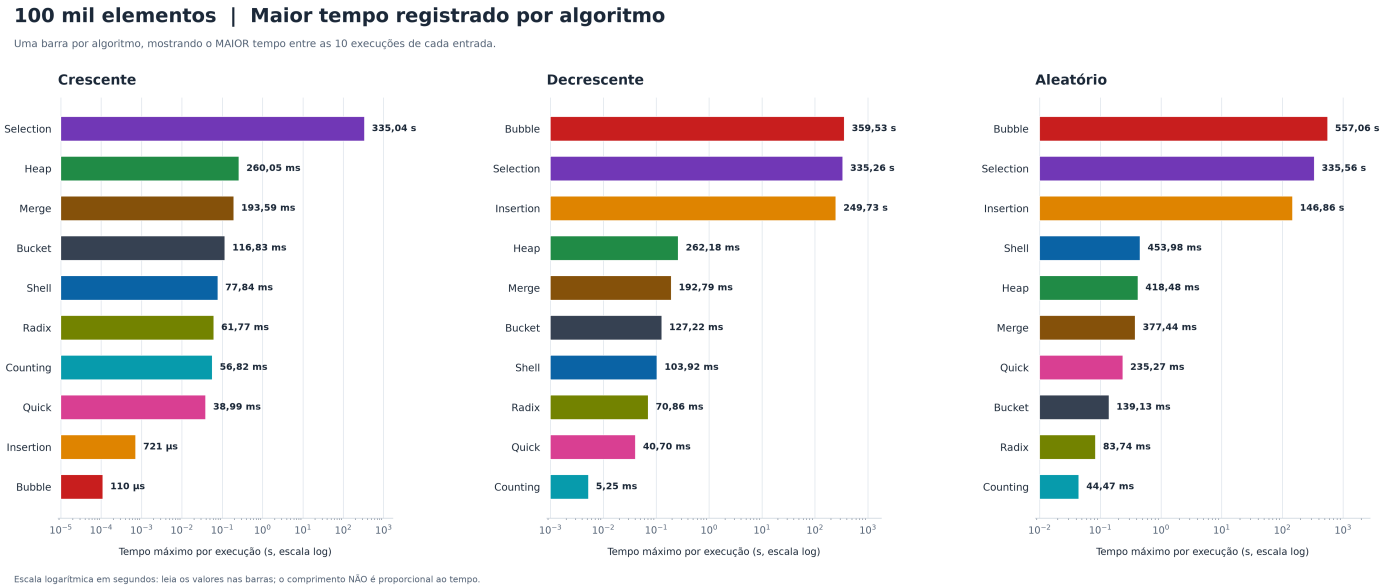


Figura 6. Tempos máximos de execução com 100 mil elementos no Computador 1.

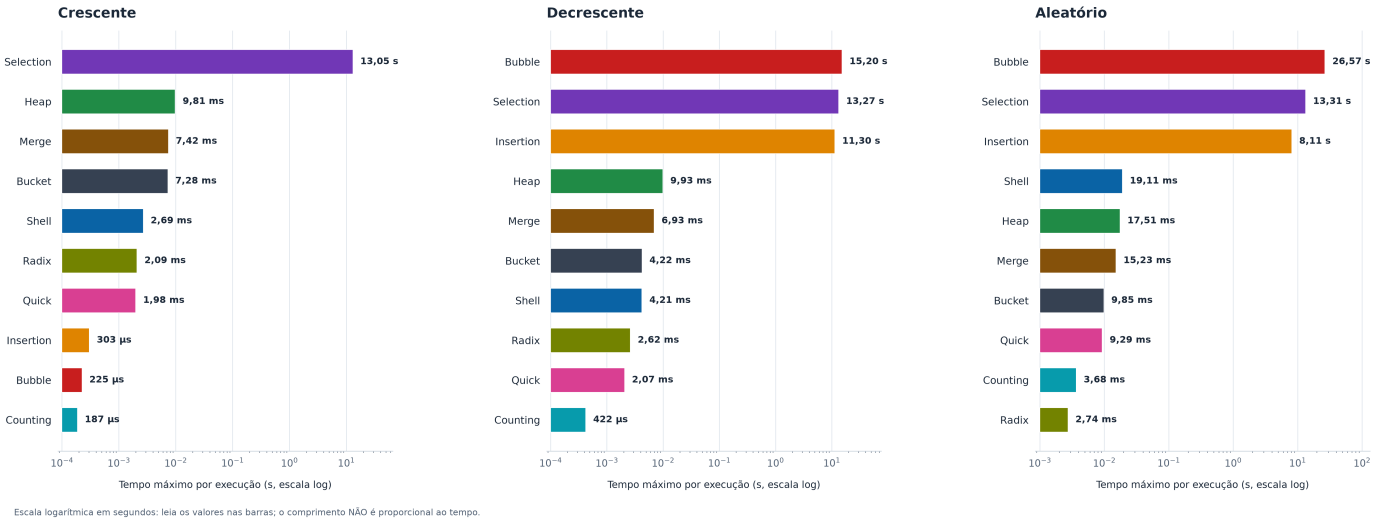


Figura 7. Tempos máximos de execução com 100 mil elementos no Computador 2.

4.3. Comparação relativa dos tempos máximos

Nessa comparação, o menor tempo máximo registrado para determinada organização de entrada é utilizado como referência ($1\times$). Cada fator indica quantas vezes o tempo máximo de outro algoritmo superou essa referência *no mesmo computador e cenário*; não representa uma razão direta entre as velocidades dos computadores.

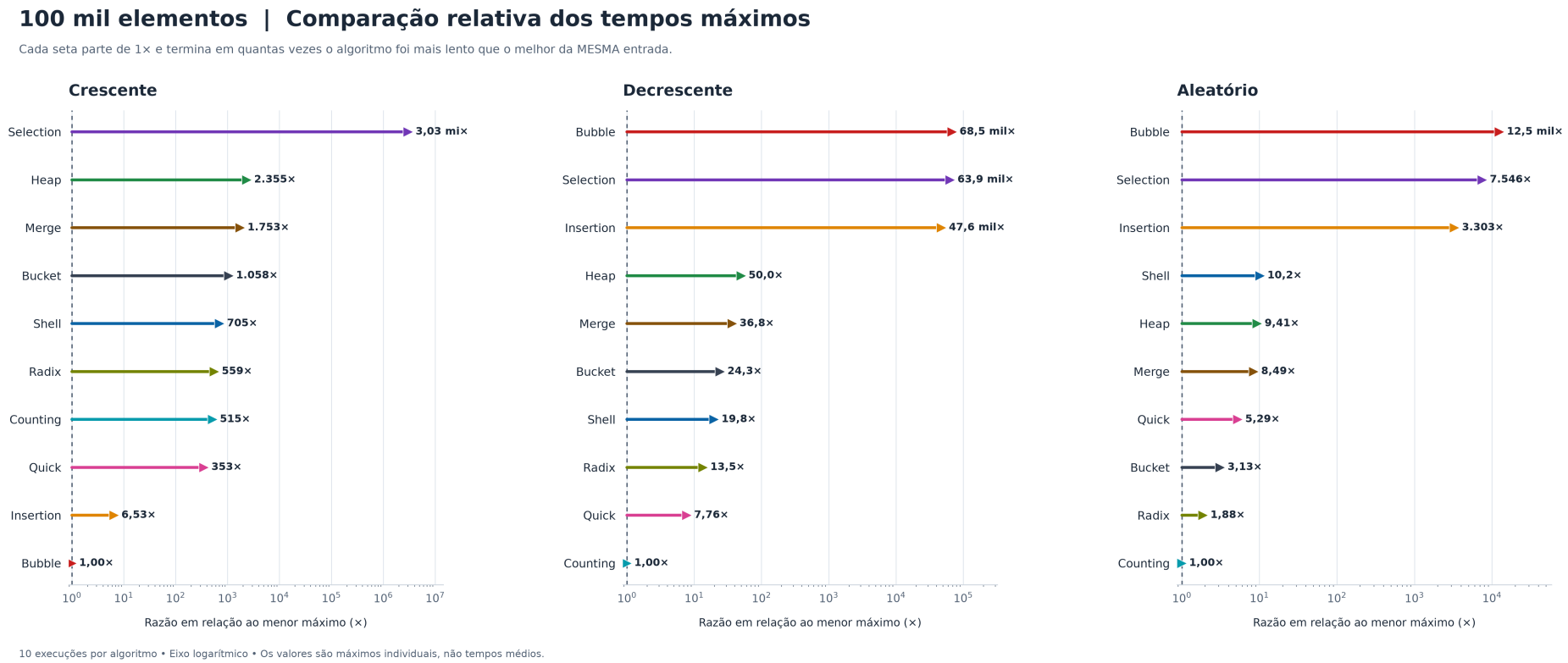


Figura 8. Fatores relativos dos tempos máximos com 100 mil elementos no Computador 1.

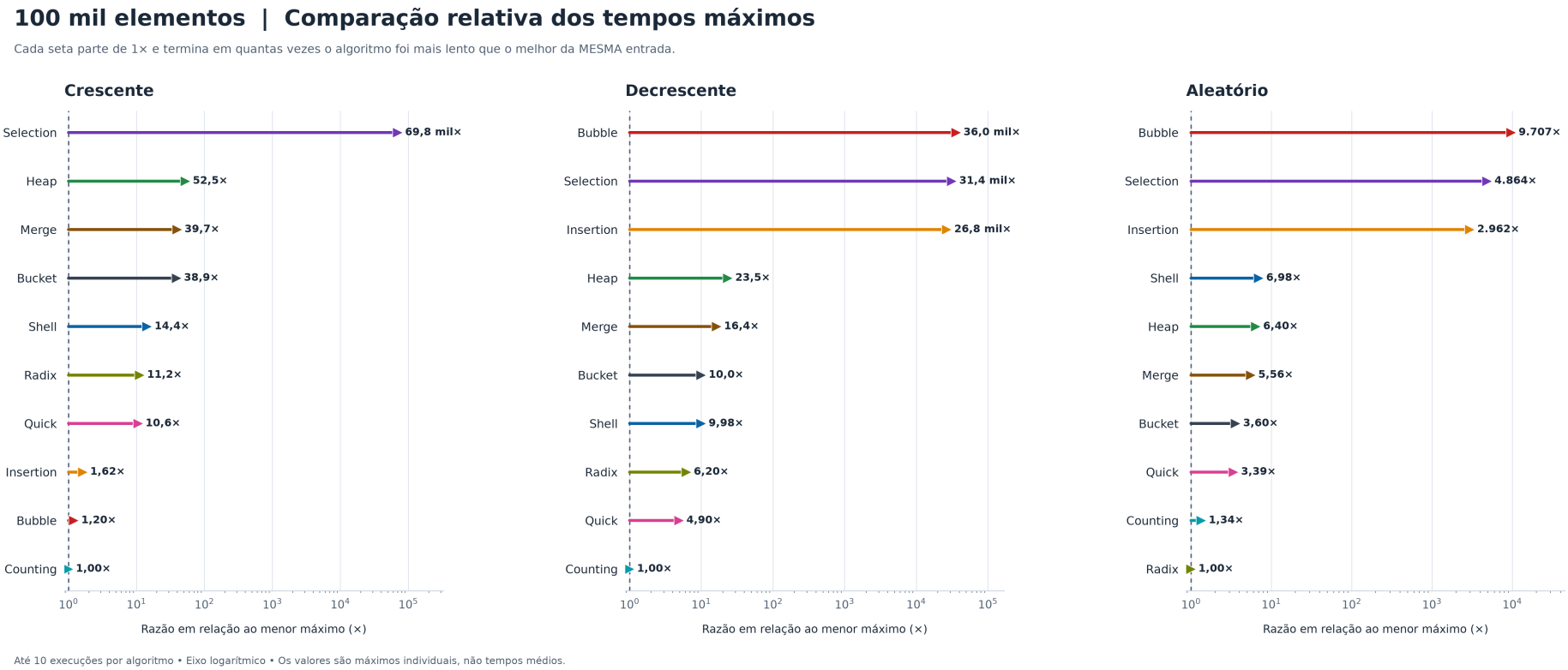


Figura 9. Fatores relativos dos tempos máximos com 100 mil elementos no Computador 2.

Na Figura 8, o Selection Sort apresentou tempo máximo cerca de 3,03 milhões de vezes maior que o Bubble Sort nos vetores crescentes. Nos vetores decrescentes e aleatórios, o Bubble Sort apresentou os maiores fatores relativos, de aproximadamente 68,5 mil e 12,5 mil vezes em relação ao Counting Sort, respectivamente.

Na Figura 9, o Selection Sort apresentou um fator próximo de 69,8 mil vezes em relação ao Counting Sort para a entrada crescente. Já o Bubble Sort apresentou fatores próximos de 36 mil vezes no cenário decrescente (referência: Counting) e de 9.707 vezes no aleatório (referência: Radix). Esses fatores se referem aos *máximos* registrados, e não às médias ou medianas.

4.4. Tempos médios com 1 milhão de elementos

Para 1 milhão de elementos, os sete algoritmos executados nos dois ambientes foram Shell, Heap, Merge, Quick, Counting, Radix e Bucket Sort. As Figuras 10 e 11 mostram os tempos médios obtidos a partir de dez execuções por configuração.

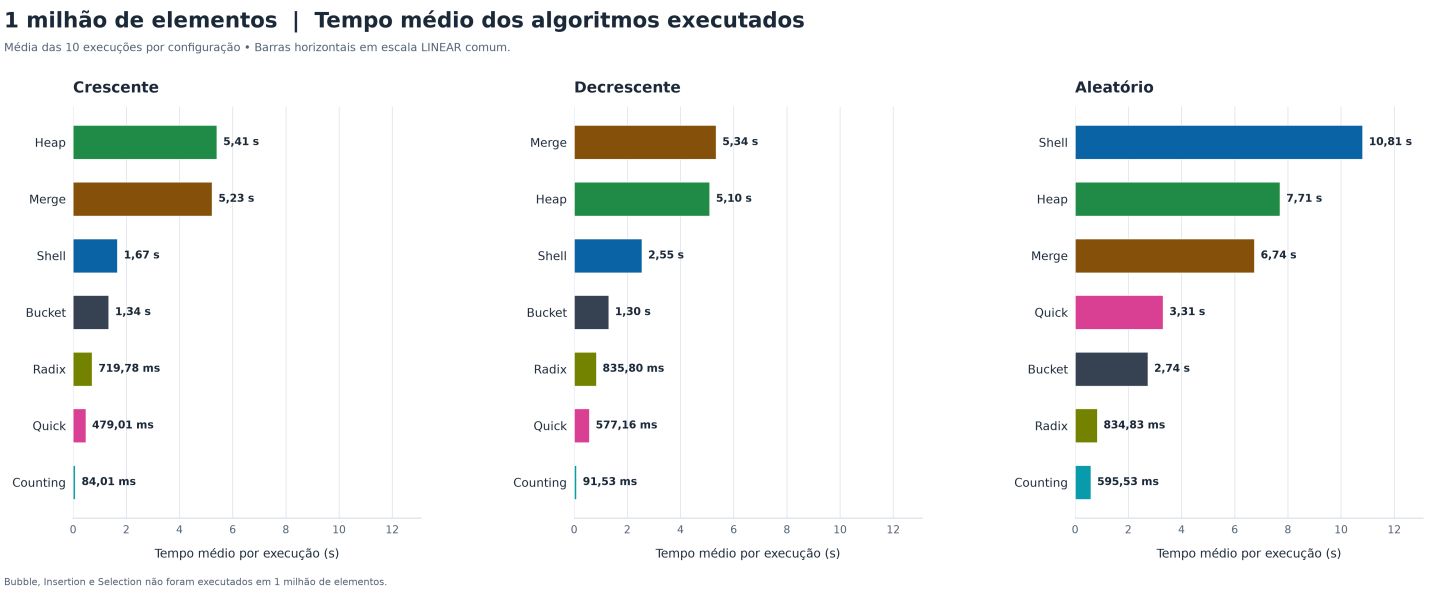


Figura 10. Tempos médios com 1 milhão de elementos para os sete algoritmos executados no Computador 1.

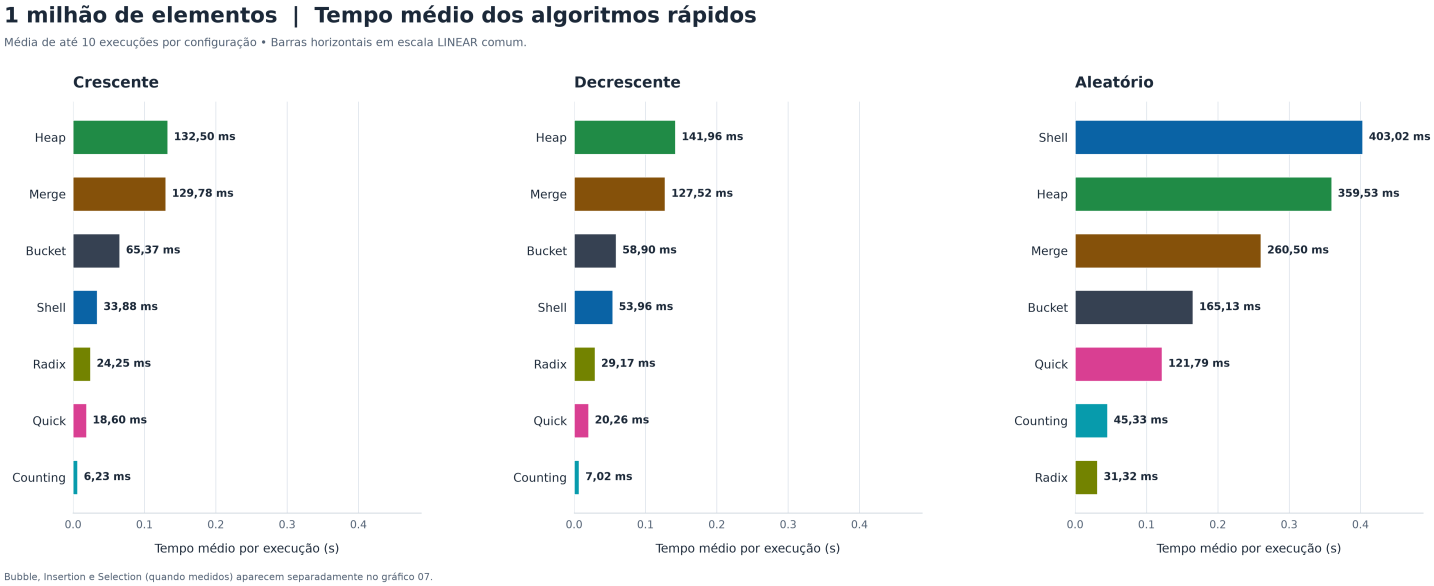


Figura 11. Tempos médios com 1 milhão de elementos para os sete algoritmos executados no Computador 2.

Em vetores crescentes, por exemplo, o Heap Sort apresentou aproximadamente 5,41 s no Computador 1 e 132,50 ms no Computador 2. O Merge Sort

registrou aproximadamente 5,23 s e 129,78 ms, respectivamente. As diferenças são expressivas, mas não isolam o efeito de um único componente de hardware.

4.4.1. Bubble, Insertion e Selection Sort no Computador 2

Devido ao elevado custo dos experimentos, Bubble Sort, Insertion Sort e Selection Sort não foram avaliados com 1 milhão de elementos no Computador 1. No Computador 2, as dez repetições dessas configurações foram registradas e estão representadas na Figura 12.

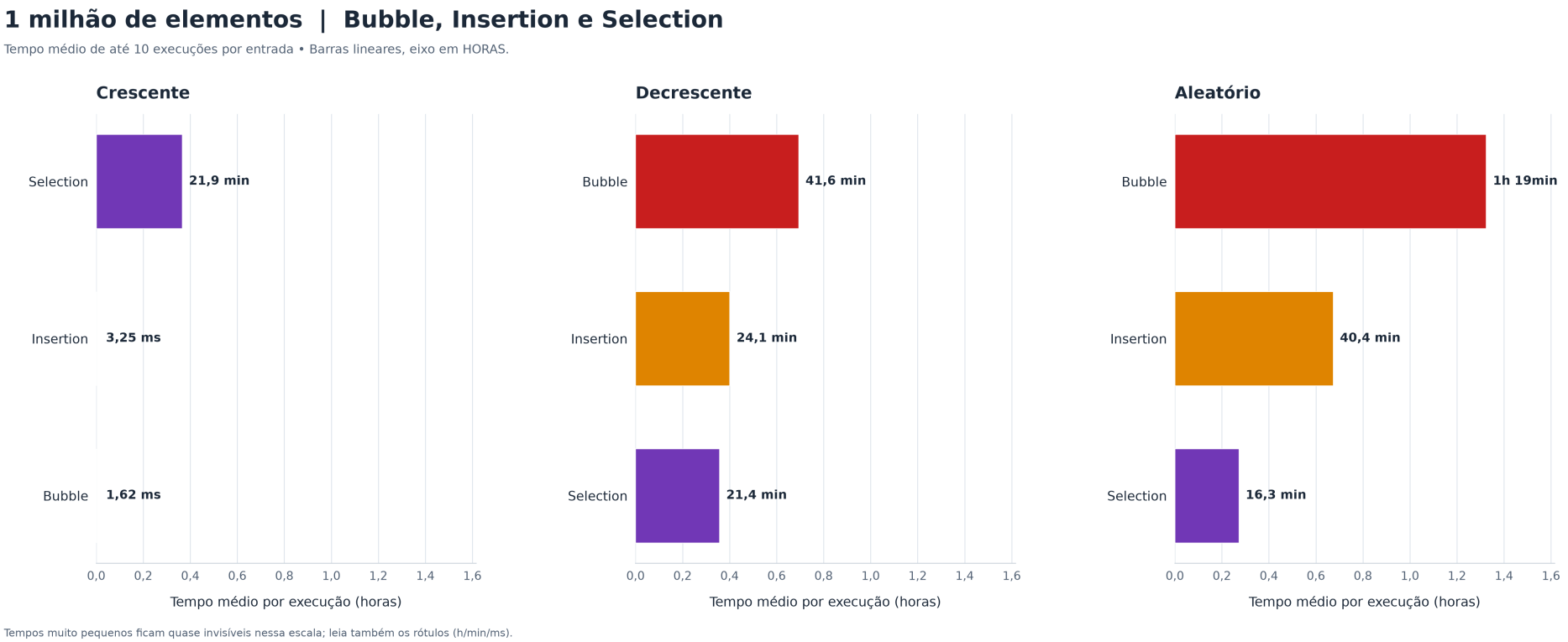


Figura 12. Tempos médios de Bubble, Insertion e Selection Sort com 1 milhão de elementos no Computador 2.

O Selection Sort levou aproximadamente 21,9 minutos para a entrada crescente, enquanto Bubble e Insertion Sort executaram em poucos milissegundos nessa configuração. Para entradas aleatórias, o Bubble Sort apresentou tempo médio próximo de 1 hora e 19 minutos e o Insertion Sort, aproximadamente 40,4 minutos. Nas entradas decrescentes, os tempos foram de aproximadamente 41,6 e 24,1 minutos, respectivamente.

Os resultados evidenciam diferenças marcantes entre algoritmos e cenários de entrada. A discussão a seguir relaciona esses comportamentos à complexidade assintótica e a aspectos específicos das implementações.

5. Análise e Discussão

Os resultados obtidos nos experimentos permitiram observar diferenças significativas no desempenho dos algoritmos de ordenação. Essas diferenças estão relacionadas principalmente à complexidade computacional, à quantidade de elementos e à organização inicial dos vetores.

Nesta seção, os resultados experimentais são relacionados às características teóricas dos algoritmos, considerando também as diferenças entre os dois ambientes de execução.

5.1. Influência da complexidade computacional

Uma das principais observações dos experimentos foi o crescimento elevado dos tempos de execução dos algoritmos de complexidade quadrática, especialmente Bubble Sort, Insertion Sort e Selection Sort.

O Selection Sort apresentou um comportamento particularmente interessante, pois seu tempo de execução permaneceu elevado mesmo quando os vetores já estavam ordenados. Isso ocorre porque o algoritmo continua realizando comparações para encontrar o menor elemento, independentemente da organização inicial dos dados.

No Computador 2, o Selection Sort apresentou cerca de 13 segundos em 100 mil elementos crescentes e aproximadamente 21,9 minutos em 1 milhão. Embora os gráficos de 100 mil e de 1 milhão apresentem estatísticas distintas (máximo e média, respectivamente), a mudança de escala observada é compatível com a complexidade $O(n^2)$. Sob essa ordem de crescimento, aumentar a entrada dez vezes pode elevar a quantidade de operações em cerca de cem vezes.

Em contrapartida, algoritmos como Heap Sort, Merge Sort e Quick Sort apresentaram crescimento consideravelmente menor. Isso está relacionado às suas complexidades típicas de $O(n \log n)$, que proporcionam maior eficiência para grandes quantidades de elementos.

Esses resultados demonstram que algoritmos com tempos semelhantes para vetores pequenos podem apresentar diferenças muito grandes conforme a quantidade de dados aumenta.

5.2. Influência da organização inicial dos vetores

A organização dos elementos também demonstrou influência significativa sobre o desempenho dos algoritmos.

Nos vetores crescentes, Bubble Sort e Insertion Sort apresentaram tempos extremamente baixos. Isso ocorre porque as implementações utilizadas conseguem aproveitar a ordenação inicial dos elementos.

No Bubble Sort, foi utilizada uma verificação que permite encerrar o algoritmo quando nenhuma troca é realizada durante uma passagem. Já no Insertion Sort, elementos previamente ordenados não precisam ser deslocados para outras posições.

No Computador 2, com 1 milhão de elementos crescentes, o Bubble Sort apresentou tempo médio de aproximadamente 1,62 milissegundo, enquanto o Insertion Sort registrou cerca de 3,25 milissegundos.

Entretanto, nos vetores decrescentes e aleatórios, esses algoritmos apresentaram um aumento expressivo no tempo de execução. O Bubble Sort, por exemplo, chegou a aproximadamente 1 hora e 19 minutos para ordenar 1 milhão de elementos aleatórios no Computador 2.

Isso demonstra que um algoritmo pode apresentar excelente desempenho em determinadas condições e resultados muito inferiores em outras.

O Shell Sort também apresentou variações entre os diferentes tipos de entrada. Sua estratégia de ordenação por intervalos permite reduzir deslocamentos, mas o desempenho continua dependendo da organização dos elementos e da sequência de intervalos utilizada.

Por outro lado, Heap Sort e Merge Sort apresentaram um crescimento mais previsível, compatível com suas complexidades assintóticas de $O(n \log n)$.

5.3. Desempenho dos algoritmos não comparativos

Os algoritmos Counting Sort, Radix Sort e Bucket Sort também apresentaram resultados relevantes, especialmente nos experimentos com maior quantidade de elementos.

O Counting Sort destacou-se por apresentar tempos reduzidos em diversas configurações. No Computador 1, por exemplo, sua média para 1 milhão de elementos crescentes foi de aproximadamente 84 milissegundos, enquanto o Quick Sort apresentou cerca de 479 milissegundos.

No Computador 2, o Counting Sort também apresentou um desempenho elevado, registrando aproximadamente 6,23 milissegundos na mesma configuração.

Entretanto, os algoritmos não comparativos possuem características e limitações específicas. O Counting Sort depende da faixa dos valores presentes no vetor, enquanto o Radix Sort realiza a ordenação por dígitos, e o Bucket Sort depende da distribuição dos elementos entre os grupos. No caso aleatório, os dados foram gerados em uma faixa proporcional a dez vezes o tamanho do vetor, o que também aumenta o custo de inicialização das estruturas auxiliares do Counting Sort.

Nos vetores aleatórios com 1 milhão de elementos, por exemplo, o Radix Sort apresentou tempo médio de aproximadamente 31,32 milissegundos no Computador 2, enquanto o Counting Sort registrou cerca de 45,33 milissegundos.

Essa diferença demonstra que a complexidade assintótica não é o único fator responsável pelo desempenho. Características dos dados, operações realizadas e utilização de memória também podem influenciar os tempos obtidos.

Além disso, nos experimentos, os algoritmos por comparação utilizaram vetores de objetos `Integer`, enquanto os algoritmos não comparativos trabalharam com vetores de inteiros primitivos `int`. Essa diferença de representação também deve ser considerada ao interpretar os resultados entre as duas categorias.

5.4. Comparação entre os computadores

Os experimentos realizados nos dois computadores apresentaram diferenças expressivas nos tempos de execução.

De maneira geral, o Computador 2 apresentou tempos menores para os algoritmos executados em ambos os ambientes.

Um exemplo foi o Heap Sort com 1 milhão de elementos crescentes, que apresentou tempo médio de aproximadamente 5,41 segundos no Computador 1 e 132,50 milissegundos no Computador 2.

Diferenças semelhantes foram observadas em outros algoritmos, como Merge Sort, Quick Sort e Shell Sort.

Entretanto, esses resultados não permitem afirmar que as diferenças sejam causadas exclusivamente pelos processadores utilizados. Outros fatores, como memória disponível, sistema operacional, gerenciamento de recursos e comportamento da Máquina Virtual Java (JVM), podem influenciar as medições.

O Computador 1 utilizou o sistema Zorin OS, enquanto o Computador 2 executou os testes no Ubuntu por meio do WSL, no Windows 11.

Dessa forma, a comparação representa o desempenho observado nos ambientes utilizados, não constituindo uma avaliação isolada da capacidade dos processadores.

5.5. Variação dos tempos e limitações experimentais

A análise exploratória dos tempos individuais e dos boxplots produzidos durante o tratamento dos dados, embora não reproduzidos integralmente neste documento, mostrou que as execuções de uma mesma configuração nem sempre apresentaram valores idênticos.

Em alguns casos, foram identificados valores isolados superiores aos demais registros. Essas variações podem estar relacionadas a fatores externos, como processos executados pelo sistema operacional, gerenciamento de memória ou atividades internas da JVM.

Entretanto, não é possível determinar a causa específica desses valores apenas com os tempos registrados.

A utilização de dez execuções por configuração permitiu observar parte dessa variabilidade, embora uma quantidade maior de repetições pudesse contribuir para uma análise estatística mais detalhada.

Também é importante considerar que a comparação relativa dos tempos máximos utiliza os maiores valores registrados, enquanto os gráficos de crescimento e de 1 milhão de elementos utilizam, respectivamente, medianas e médias.

Portanto, essas medidas devem ser interpretadas de acordo com seus objetivos, evitando tratar valores máximos isolados como representativos de todas as execuções.

Apesar dessas limitações, os experimentos permitiram identificar tendências consistentes com os comportamentos esperados dos algoritmos estudados.

6. Conclusão

O desenvolvimento deste trabalho permitiu analisar o desempenho de dez algoritmos de ordenação, relacionando seus comportamentos práticos às complexidades computacionais estudadas teoricamente.

Por meio dos experimentos, foi possível observar que o aumento da quantidade de elementos influencia diretamente o tempo de execução, mas essa influência ocorre de maneiras diferentes para cada algoritmo.

Os resultados demonstraram que algoritmos de complexidade quadrática, como Bubble Sort, Insertion Sort e Selection Sort, podem apresentar tempos elevados para grandes quantidades de dados. Entretanto, também foi observado que Bubble Sort e Insertion Sort podem apresentar bom desempenho quando os vetores já estão ordenados.

Por outro lado, algoritmos como Heap Sort, Merge Sort e Quick Sort apresentaram maior eficiência para entradas maiores, enquanto os algoritmos não comparativos também obtiveram resultados expressivos quando utilizados em condições adequadas.

A utilização de dois computadores permitiu observar diferenças significativas nos tempos registrados, evidenciando que o desempenho experimental não depende exclusivamente da complexidade teórica, mas também das características da implementação e do ambiente de execução.

Dessa forma, conclui-se que não existe um único algoritmo de ordenação que seja a melhor escolha para todas as situações. A seleção deve considerar fatores como quantidade de elementos, organização dos dados, características dos valores e recursos computacionais disponíveis.

Por fim, os experimentos contribuíram para consolidar o entendimento sobre os algoritmos de ordenação, demonstrando a importância de combinar o estudo teórico com testes práticos para compreender suas vantagens, limitações e aplicações.

Referências

[1] CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Introduction to Algorithms*. 4. ed. MIT Press, 2022.

[2] SEDGEWICK, R.; WAYNE, K. *Algorithms*. 4. ed. Addison-Wesley, 2011.

[3] ORACLE. *Java SE 25: System Class Documentation*. Disponível em: `https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/System.html`. Acesso em: 27 out. 2026.